

G10_Task

Hello,

Please read the following carefully:

As a Software Engineering research team at the Polytechnique Montréal (Canada), we are interested in understanding developers' assessment of coding task difficulty. You will be provided with a series of 12 coding questions. Questions are divided into two parts:

- First, you will be given three instructions representing coding tasks and you will be asked to write, in a few words, what is the underlying task represented by all these instructions. The instructions represent the same tasks, but they vary in wording and information they give. In a nutshell, you have to distill the essence of the task and write it in a few words.

- In a second time, you will be asked to assess how difficult implementing this **task** would be. Note we ask the assessment based on the task you will have worded in the previous part, not based on any of the instructions presented earlier. First, you will be asked your subjective judgment on a scale from Very Easy to Very Difficult. Then, to make judging the difficulty more relatable, we adopt a similar approach as what is used in agile development and planning poker, i.e. you will be asked to evaluate how long it would take to implement a task (in minutes here). Consider in your assessment the knowledge needed to implement the task, the external information you would need to use, whether you would need the assistance of a LLM such as ChatGPT (and so the time it would take to debug the obtained code) etc. Finally, you will have to explain in a few words why you gave such difficulty ratings.

To give you an idea of the range of tasks you will need to assess, the first section of this questionnaire will be two examples we devised to illustrate the questionnaire.

Completing the task will take about 30 minutes.

Note also that all examples are written using Python 3.10 syntax.

Please make sure to answer all the questions to the best of your ability.

Please note that your identity and personal data will not be disclosed, while we plan to use the aggregated results and anonymized responses as part of a scientific publication. If you have any questions about the questionnaire or our research, please do not hesitate to contact us.

By proceeding, you consent to take part in the study under the conditions described previously.

Thank you,

Florian Tambon (florian-2.tambon@polymtl.ca)
Cyrine Zid (cyrine.zid@polymtl.ca)
Amin Nikanjam (amin.nikanjam@polymtl.ca)
Foutse Khomh (foutse.khomh@polymtl.ca)
Giuliano Antoniol (giuliano.antoniol@polymtl.ca)

* Indique une question obligatoire

1. Please enter your prolific ID : *

Example of tasks

As we ask of you some estimation of difficulty and the tasks dealt in this questionnaire can be a bit different from what you can be used to as a developer, we give you examples of some tasks along with some assessments.

Note that this assessment is **our assessment**. You may have a different opinion based on your background. The idea of those examples is to give an extreme range of what type of tasks you can expect in the questionnaire.

As a reminder: You will first be given three instructions and asked to explain what the task is about in a few words. Then, you will have to rate the difficulty of the **task** you just described (not accounting for particular instruction). Rating the difficulty is done in two steps: subjective difficulty and approximate time needed. Finally, you will be asked to say why you gave such rating.

Example 1

Instruction 1

```
1 def subtract(c1, c2):  
2     """  
3     Deliver the result of subtracting two complex numbers,  
4     specified as 'c1' and 'c2', as a complex output.  
5     :param c1: The first complex number, complex.  
6     :param c2: The second complex number, complex.  
7     :return: The difference of the two complex numbers, complex.  
8     """
```

Instruction 2

```
1 def subtract(c1, c2):
2     """
3     The method receives two complex numbers 'c1' and 'c2', from which it
4     subtracts 'c2.real' from 'c1.real' to determine the real portion, and
5     subtracts 'c2.imag' from 'c1.imag' to determine the imaginary portion.
6     It constructs and returns a new complex number using these results
7     with the 'complex()' function.
8     :param c1: The first complex number, complex.
9     :param c2: The second complex number, complex.
10    :return: The difference of the two complex numbers, complex.
11    """
```

Instruction 3

```
1 def subtract(c1, c2):
2     """
3     Subtracts two complex numbers.
4     :param c1: The first complex number, complex.
5     :param c2: The second complex number, complex.
6     :return: The difference of the two complex numbers, complex.
7     >>> complexCalculator = ComplexCalculator()
8     >>> complexCalculator.subtract(1+2j, 3+4j)
9     (-2-2j)
10    """
```

Answer

The task encapsulated by the **subtract**

function

is to compute the subtraction between two complex numbers.

This task has a subjective difficulty of **Very Easy** as assessed by us and would take about **1 min**

to implement. There is no particular difficulty, the task is very straightforward without any corner case, it just requires to know what a complex is, and the standard Python library for it.

Example 2

Context

```
1 import numpy as np
2 class MetricsCalculator2:
3     """
4     The class provides to calculate Mean Reciprocal Rank (MRR) and Mean Average Precision (MAP)
5     based on input data, where MRR measures the ranking quality and MAP measures the average precision.
6     """
7
8     def __init__(self):
9         pass
10
11     @staticmethod
12     def map(data):
13         pass
```

Instruction 1

```
1 def mrr(data):
2     """
3     Compute the MRR of the input data. MRR is a widely used evaluation index.
4     It is the mean of reciprocal rank.
5     :param data: the data must be a tuple, list 0,1, eg. ([1,0,...],5).
6     In each tuple (actual result,ground truth num),ground truth num is the total ground num.
7     ([1,0,...],5), or list of tuple eg. [([1,0,1,...],5),([1,0,...],6),([0,0,...],5)].
8     1 stands for a correct answer, 0 stands for a wrong answer.
9     :return: if input data is list, return the recall of this list. if the input data is list of list, return the
10     average recall on all list. The second return value is a list of precision for each input.
11     >>> MetricsCalculator2.mrr([([1, 0, 1, 0], 4)])
12     >>> MetricsCalculator2.mrr([([1, 0, 1, 0], 4), ([0, 1, 0, 1], 4)])
13     1.0, [1.0]
14     0.75, [1.0, 0.5]
15
16     """
```

Instruction 2

```
1 def mrr(data):
2     """
3     Assess the Mean Reciprocal Rank (MRR) from the input 'data'.
4     MRR assesses the average of reciprocal ranks of outcomes.
5     The input structure 'data' must be a tuple consisting of a list of
6     binary values and its total count or a list of such tuples.
7     Each binary value (0 or 1) represents the correctness of an answer.
8     If 'data' is a tuple, the function returns its mean reciprocal rank.
9     If 'data' is a list, the function returns the overall average MRR along
10     with a list showing reciprocal ranks for each component tuple.
11
12     :param data: the data must be a tuple, list 0,1, eg. ([1,0,...],5).
13     In each tuple (actual result,ground truth num),ground truth num is the total ground num.
14     ([1,0,...],5), or list of tuple eg. [([1,0,1,...],5),([1,0,...],6),([0,0,...],5)].
15     1 stands for a correct answer, 0 stands for a wrong answer.
16     :return: if input data is list, return the recall of this list. if the input data is list of list, return the
17     average recall on all list. The second return value is a list of precision for each input.
18
19     """
```

Instruction 3

```
1 def mrr(data):
2     """
3     Evaluate the Mean Reciprocal Rank (MRR) from 'data'. This metric calculates the mean of reciprocal
4     ranks for correct answers. Input 'data' could be either a tuple containing binary correctness indicators
5     and the aggregate count of answers, or multiple tuples of this type in a list. For each tuple, the function
6     measures reciprocal ranks by multiplying the binary indicators by reciprocals of their positions, finding the
7     earliest nonzero value to establish the rank of the first correct response. If the input 'data' is a tuple,
8     the MRR calculated for this specific tuple is returned; if it's a list, the function summarizes by providing
9     both the average MRR and a list of individual MRR values for each tuple.
10
11     :param data: the data must be a tuple, list 0,1, eg. ([1,0,...],5).
12     In each tuple (actual result,ground truth num),ground truth num is the total ground num.
13     ([1,0,...],5), or list of tuple eg. [([1,0,1,...],5),([1,0,...],6),([0,0,...],5)].
14     1 stands for a correct answer, 0 stands for a wrong answer.
15     :return: if input data is list, return the recall of this list. if the input data is list of list, return the
16     average recall on all list. The second return value is a list of precision for each input.
17
18     """
```

Answer

The task encapsulated by the **mrr** function is to compute the Mean Reciprocal Rank of binary tuples data.

This task has a subjective difficulty of **Very Hard** as assessed by us and would take about **40 min**

to implement. The difficulty mainly branches from dealing with the tuple data and how the formula is processed accordingly (with a few corner cases). This can take additional time if the Mean Reciprocal Rank is not known.

Question 1:

Below are instructions for implementing the function.

Instruction 1

```
1 def same_chars(s0: str, s1: str):
2     """
3     Check if two words have the same characters.
4     >>> same_chars('eabcdzzzz', 'dddzzzzzzzddeddabc')
5     True
6     >>> same_chars('abcd', 'dddddddabc')
7     True
8     >>> same_chars('dddddddabc', 'abcd')
9     True
10    >>> same_chars('eabcd', 'dddddddabc')
11    False
12    >>> same_chars('abcd', 'dddddddabce')
13    False
14    >>> same_chars('eabcdzzzz', 'dddzzzzzzzddddabc')
15    False
16    """
```

Instruction 2

```
1 def same_chars(s0: str, s1: str):
2     """
3     Construct a function 'same_chars' that assesses
4     if the characters in two input words are identical.
5     """
```

Instruction 3

```
1 def same_chars(s0: str, s1: str):
2     """
3     Write a function named 'same_chars' that checks
4     if two given words consist of the same characters.
5     """
```

2. In a few words, what is the task this function aims to implement? *

3. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ☐ Very Easy
- ☐ Easy
- ☐ Not Easy Nor Difficult
- ☐ Difficult
- ☐ Very Difficult

4. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- ☐ 5 min
- ☐ 10 min
- ☐ 20 min
- ☐ 40 min
- ☐ > 40 min

5. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 2:

Below are instructions for implementing the function:

Context

```
1 import re
2 class SplitSentence:
3     """
4     The class allows to split sentences, count words in a sentence,
5     and process a text file to find the maximum word count.
6     """
7
8     def count_words(self, sentence):
9         pass
10
11    def process_text_file(self, sentences_string):
12        pass
13
14
15
16
```

Instruction 1

```
1 def split_sentences(self, sentences_string):
2     """
3     Split a string into a list of sentences. Sentences end with .
4     or ? and with a space after that. Please note that Mr.
5     also end with . but are not sentences.
6     :param sentences_string: string, string to split
7     :return: list, split sentence list
8     >>> ss = SplitSentence()
9     >>> ss.split_sentences("aaa aaaa. bb bbbb bbb? cccc cccc. dd ddd?")
10    ['aaa aaaa.', 'bb bbbb bbb?', 'cccc cccc.', 'dd ddd?']
11    """
```


Instruction 2

```
1 def split_sentences(self, sentences_string):
2     """
3     Segment the 'sentences_string' into a sequence
4     of sentences using punctuation marks (periods or
5     question marks followed by a space) as sentence
6     terminators. Produce and return a list of these
7     sentences, being careful to overlook periods used
8     in abbreviations such as 'Inc.' which should not
9     signal the conclusion of a sentence.
10    :param sentences_string: string, string to split
11    :return:list, split sentence list
12    """
```

Instruction 3

```
1 def split_sentences(self, sentences_string):
2     """
3     Split the provided "sentences_string" into a list
4     of sentences based on specific punctuation marks.
5     Return these split sentences as a list. Sentences
6     should end with a period (.) or a question mark (?)
7     followed by a space, but exceptions exist where a
8     period is part of an abbreviation (e.g., "Mr.")
9     and should not be considered the end of a sentence.
10    :param sentences_string: string, string to split
11    :return:list, split sentence list
12    """
```

6. In a few words, what is the task this function aims to implement? *

7. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ☐ Very Easy
- ☐ Easy
- ☐ Not Easy Nor Difficult
- ☐ Difficult
- ☐ Very Difficult

8. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- ☐ 1 min
- ☐ 5 min
- ☐ 10 min
- ☐ 20 min
- ☐ 40 min
- ☐ > 40 min

9. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 3:

Below are instructions for implementing the function:

Instruction 1

```
1 def digits(n):  
2     """  
3     Given a positive integer n,  
4     return the product of the odd digits.  
5     Return 0 if all digits are even.  
6     For example:  
7     digits(1) == 1  
8     digits(4) == 0  
9     digits(235) == 15  
10    """
```

Instruction 2

```
1 def digits(n):  
2     """  
3     Create a function named 'digits' which determines  
4     the product of the odd numerical digits of a provided  
5     positive integer n, returning 0 if odd digits are absent.  
6     """
```

Instruction 3

```
1 def digits(n):  
2     """  
3     Define a function called 'digits' that multiplies all  
4     the odd digits in a positive integer n. The function  
5     returns 0 if there are no odd digits.  
6     """
```

10. In a few words, what is the task this function aims to implement? *

11. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ☐ Very Easy
- ☐ Easy
- ☐ Not Easy Nor Difficult
- ☐ Difficult
- ☐ Very Difficult

12. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- ☐ 1 min
- ☐ 5 min
- ☐ 10 min
- ☐ 20 min
- ☐ 40 min
- ☐ > 40 min

13. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 4:

Below are instructions for implementing the function:

Context

```
1 import logging
2 import datetime
3
4 class AccessGatewayFilter:
5     """
6     This class is a filter used for accessing gateway filtering,
7     primarily for authentication and access log recording.
8     """
9     def __init__(self):
10         pass
11
12     def filter(self, request):
13         pass
14
15
16     def is_start_with(self, request_uri):
17         pass
18
19     def set_current_user_info_and_log(self, user):
20         pass
21
22
```

Instruction 1

```
1 def get_jwt_user(self, request):
2     """
3     Get the user information from the JWT token in the request.
4     :param request: dict, the incoming request details
5     :return: dict or None, the user information if the token is
6     valid, None otherwise
7     >>> filter = AccessGatewayFilter()
8     >>> filter.get_jwt_user({'headers': {'Authorization': {'user':
9         {'name': 'user1'}, 'jwt': 'user1'+str(datetime.date.today())}}})
10    {'user': {'name': 'user1'}}
```

Instruction 2

```
1 def get_jwt_user(self, request):
2     """
3     Extract user details from a JWT contained within the
4     'request' headers, initially by getting the token using
5     'request['headers']['Authorization']'. The function
6     subsequently verifies whether the JWT begins with
7     the user's name, which is accessed via 'token['user']['name']'.
8     If confirmed, it separates the date from the JWT utilizing
9     the user's name to split it, achieving 'jwt_str_date'.
10    This string date is then transformed into a datetime
11    object termed 'jwt_date' employing Python's datetime
12    library formatted as '%Y-%m-%d'. Should the time span
13    between today and 'jwt_date' not exceed three days, the
14    function returns the user details in 'token'; otherwise,
15    it yields 'None' indicating an expired or invalid token.
16    :param request: dict, the incoming request details
17    :return: dict or None, the user information if the token
18            is valid, None otherwise
19    """
```

Instruction 3

```
1 def get_jwt_user(self, request):
2     """
3     Get user information from a JWT token found in the "request" headers.
4     Begin by retrieving the token from "request['headers']['Authorization']".
5     The function then checks if the JWT token starts with the user's name,
6     extracted as "token['user']['name']". If it does, the function extracts
7     the date segment from the JWT using the user's name as a splitter,
8     resulting in "jwt_str_date". This string is then converted into a
9     datetime object, "jwt_date", using Python's datetime module with the
10    format "%Y-%m-%d". If the difference between the current date
11    and "jwt_date" is three days or less, it returns the user details
12    found in "token"; otherwise, it returns "None" to indicate that
13    the token is either invalid or expired.
14    :param request: dict, the incoming request details
15    :return: dict or None, the user information if the token is valid, None otherwise
16    """
```

14. In a few words, what is the task this function aims to implement? *

15. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ☐ Very Easy
- ☐ Easy
- ☐ Not Easy Nor Difficult
- ☐ Difficult
- ☐ Very Difficult

16. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- ☐ 1 min
- ☐ 5 min
- ☐ 10 min
- ☐ 20 min
- ☐ 40 min
- ☐ > 40 min

17. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 5:

Below are instructions for implementing the function:

Instruction 1

```
1 def file_name_check(file_name):
2     """
3     Create a function which takes a string representing a file's name, and returns
4     'Yes' if the the file's name is valid, and returns 'No' otherwise.
5     A file's name is considered to be valid if and only if all the following conditions
6     are met:
7     - There should not be more than three digits ('0'-'9') in the file's name.
8     - The file's name contains exactly one dot '.'
9     - The substring before the dot should not be empty, and it starts with a letter from
10    the latin alphapet ('a'-'z' and 'A'-'Z').
11    - The substring after the dot should be one of these: ['txt', 'exe', 'dll']
12    Examples:
13    file_name_check("example.txt") # => 'Yes'
14    file_name_check("1example.dll") # => 'No' (the name should start with a latin
15                                         alphapet letter)
16    """
```

Instruction 2

```
1 def file_name_check(file_name):
2     """
3     Develop a function named 'file_name_check' to determine the validity
4     of a file name based on defined criteria, returning 'Yes' for valid
5     names and 'No' for invalid ones. The name must not exceed three digits,
6     should include only one dot, start with a letter before the dot without
7     being empty, and end in one of these extensions: 'txt', 'exe', or 'dll'.
8     """
```

Instruction 3

```
1 def file_name_check(file_name):
2     """
3     Construct a function named 'file_name_check' which takes a string
4     indicating the file name as a parameter. This function should return
5     'Yes' if the file's name matches specific valid conditions, else 'No'.
6     The file name must not possess over three digits, should contain only
7     one dot dividing it into two portions, the first portion must begin
8     with a letter and shall not be blank, while the last portion should
9     be one of the specified extensions ('txt', 'exe', 'dll'). It should
10    first check for the number of digits, dot existence, check if the
11    first part isn't empty and starts with a letter, finally confirming
12    the extension matches.
13    """
```


18. In a few words, what is the task this function aims to implement? *

19. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ☐ Very Easy
- ☐ Easy
- ☐ Not Easy Nor Difficult
- ☐ Difficult
- ☐ Very Difficult

20. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- ☐ 1 min
- ☐ 5 min
- ☐ 10 min
- ☐ 20 min
- ☐ 40 min
- ☐ > 40 min

21. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 6:

Below are instructions for implementing the function:

Context

```
1 import numpy as np
2 from gensim import matutils
3 from numpy import dot, array
4 class VectorUtil:
5     """
6     The class provides vector operations, including calculating similarity,
7     cosine similarities, average similarity, and IDF weights.
8     """
9
10    def similarity(vector_1, vector_2):
11        pass
12
13    @staticmethod
14    def cosine_similarities(vector_1, vectors_all):
15        pass
16
17    @staticmethod
18    def compute_idf_weight_dict(total_num, number_dict):
19        pass
20
21
22    @staticmethod
23    def n_similarity(vector_list_1, vector_list_2):
24
25
```

Instruction 1

```
1 def n_similarity(vector_list_1, vector_list_2):
2     """
3     Compute cosine similarity between two sets of vectors.
4     :param vector_list_1: list of numpy vector
5     :param vector_list_2: list of numpy vector
6     :return: numpy.ndarray, Similarities between vector_list_1 and vector_list_2.
7     >>> vector_list1 = [np.array([1, 2, 3]), np.array([4, 5, 6])]
8     >>> vector_list2 = [np.array([7, 8, 9]), np.array([10, 11, 12])]
9     >>> VectorUtil.n_similarity(vector_list1, vector_list2)
10    0.9897287473881233
11    """
```

Instruction 2

```
1 def n_similarity(vector_list_1, vector_list_2):
2     """
3     Compute the cosine similarity between two sets of vectors
4     "vector_list_1" and "vector_list_2". The function should
5     return a numerical score indicating the similarity between
6     the two lists of vectors.
7     :param vector_list_1: list of numpy vector
8     :param vector_list_2: list of numpy vector
9     :return: numpy.ndarray, Similarities between vector_list_1 and vector_list_2.
10    """
```

Instruction 3

```
1 def n_similarity(vector_list_1, vector_list_2):
2     """
3     Assess the cosine similarity between 'vector_list_1' and 'vector_list_2',
4     providing a numeric similarity score. The function evaluates emptiness
5     in any list, raising a 'ZeroDivisionError' if found empty. If non-empty,
6     the average of the vectors in each list is computed, followed by the cosine
7     similarity calculation between these averages.
8     :param vector_list_1: list of numpy vector
9     :param vector_list_2: list of numpy vector
10    :return: numpy.ndarray, Similarities between vector_list_1 and vector_list_2.
11    """
```

22. In a few words, what is the task this function aims to implement? *

23. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ☐ Very Easy
- ☐ Easy
- ☐ Not Easy Nor Difficult
- ☐ Difficult
- ☐ Very Difficult

24. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- ☐ 1 min
- ☐ 5 min
- ☐ 10 min
- ☐ 20 min
- ☐ 40 min
- ☐ > 40 min

25. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 7:

Below are instructions for implementing the function:

Instruction 1

```
1 def is_palindrome(text: str):
2     """
3     Checks if given string is a palindrome
4     >>> is_palindrome('')
5     True
6     >>> is_palindrome('aba')
7     True
8     >>> is_palindrome('aaaaa')
9     True
10    >>> is_palindrome('zbcd')
11    False
12    """
```

Instruction 2

```
1 def is_palindrome(text: str):
2     """
3     Write a function named "is_palindrome" which takes a single
4     parameter "text" which is a string. The function checks if
5     "text" is identical to its reverse (which can be obtained
6     by slicing with "text[::-1]"). It returns True if the string
7     is a palindrome, otherwise it returns False.
8     """
```

Instruction 3

```
1 def is_palindrome(text: str):
2     """
3     Develop a function 'is_palindrome' accepting a string input
4     'text'. It checks whether 'text' is the same forwards and
5     backwards (reverse obtained via 'text[::-1]'). If so, it
6     returns True indicating the string is a palindrome; otherwise,
7     it returns False.
8     """
```

26. In a few words, what is the task this function aims to implement? *

27. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ☐ Very Easy
- ☐ Easy
- ☐ Not Easy Nor Difficult
- ☐ Difficult
- ☐ Very Difficult

28. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- ☐ 1 min
- ☐ 5 min
- ☐ 10 min
- ☐ 20 min
- ☐ 40 min
- ☐ > 40 min

29. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 8:

Below are instructions for implementing the function:

Context

```
1 class SQLGenerator:
2     """
3     This class generates SQL statements for common operations on a table,
4     such as SELECT, INSERT, UPDATE, and DELETE.
5     """
6
7     def __init__(self, table_name):
8         """
9         Initialize the table name.
10        :param table_name: str
11        """
12        self.table_name = table_name
13
14    def select(self, fields=None, condition=None):
15        pass
16
17    def insert(self, data):
18        pass
19
20    def update(self, data, condition):
21        pass
22
23    def delete(self, condition):
24        pass
25
26    def select_female_under_age(self, age):
27        pass
```

Instruction 1

```
1 def select_by_age_range(self, min_age, max_age):
2     """
3     Generates a SQL statement to select records within
4     a specified age range.
5     :param min_age: int. The minimum age.
6     :param max_age: int. The maximum age.
7     :return: str. The generated SQL statement.
8     >>> sql.select_by_age_range(20, 30)
9     'SELECT * FROM table1 WHERE age BETWEEN 20 AND 30;'
10    """
```

Instruction 2

```
1 def select_by_age_range(self, min_age, max_age):
2     """
3     Assemble a SQL statement designed to filter
4     records within a designated age range specified
5     by the parameters 'min_age' and 'max_age'.
6     Configure the condition for the SQL query by
7     incorporating these values into a string that
8     defines the age range for the SQL WHERE clause.
9     Subsequently, use this condition to create and
10    output the entire SQL SELECT statement.
11    :param min_age: int. The minimum age.
12    :param max_age: int. The maximum age.
13    :return: str. The generated SQL statement.
14    """
```

Instruction 3

```
1 def select_by_age_range(self, min_age, max_age):
2     """
3     Craft an SQL query to retrieve rows that fall
4     within a defined age bracket, specified by
5     'min_age' and 'max_age'. Output this query as
6     a string. The function forms a conditional SQL
7     clause 'age BETWEEN {min_age} AND {max_age}' using
8     these parameters 'min_age' and 'max_age'. By passing
9     this condition to the class's 'select' function, it
10    composes the complete SQL SELECT command. The method
11    returns this command, which chooses all table columns
12    for records with ages between the designated minimum
13    and maximum values.
14    :param min_age: int. The minimum age.
15    :param max_age: int. The maximum age.
16    :return: str. The generated SQL statement.
17    """
```

30. In a few words, what is the task this function aims to implement? *

31. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ☐ Very Easy
- ☐ Easy
- ☐ Not Easy Nor Difficult
- ☐ Difficult
- ☐ Very Difficult

32. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- ☐ 1 min
- ☐ 5 min
- ☐ 10 min
- ☐ 20 min
- ☐ 40 min
- ☐ > 40 min

33. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 9:

Below are instructions for implementing the function:

Instruction 1

```
1 def unique_digits(x):
2     """
3     Given a list of positive integers x.
4     return a sorted list of all
5     elements that hasn't any even digit.
6
7     Note: Returned list should be sorted in
8     increasing order.
9
10    For example:
11    >>> unique_digits([15, 33, 1422, 1])
12    [1, 15, 33]
13    >>> unique_digits([152, 323, 1422, 10])
14    []
15    """
```

Instruction 2

```
1 def unique_digits(x):
2     """
3     Design a function titled 'unique_digits' that extracts
4     and sorts integers from an input list, with the condition
5     that each integer contains only odd digits.
6     """
```

Instruction 3

```
1 def unique_digits(x):
2     """
3     Write the function 'unique_digits', receiving a list 'x' of
4     positive integers. The function should produce a sorted list
5     of figures from 'x' that do not include any even-numbered digits.
6     Encapsulated within this function is 'judge()', a function
7     checking each integer by transforming it into a string and
8     sequentially analyzing each character. If a digit is even when
9     interpreted as an integer, 'judge()' will return False. If every
10    digit sustains the odd condition, it returns True. Post 'judge()'
11    application, the main function filters 'x', sorts it, and presents
12    the sequence.
13    """
```

34. In a few words, what is the task this function aims to implement? *

35. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ☐ Very Easy
- ☐ Easy
- ☐ Not Easy Nor Difficult
- ☐ Difficult
- ☐ Very Difficult

36. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- ☐ 1 min
- ☐ 5 min
- ☐ 10 min
- ☐ 20 min
- ☐ 40 min
- ☐ > 40 min

37. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 10:

Below are instructions for implementing the function:

Context

```
1 class HRManagementSystem:
2     """
3     This is a class as personnel management system that implements functions
4     such as adding, deleting, querying, and updating employees
5     """
6
7     def __init__(self):
8         """
9         Initialize the HRManagementSystem withan attribute employees, which
10        is an empty dictionary.
11        """
12        self.employees = {}
13
14    def add_employee(self, employee_id, name, position, department, salary):
15        pass
16
17    def update_employee(self, employee_id: int, employee_info: dict):
18        pass
19
20    def get_employee(self, employee_id):
21        pass
22
23    def list_employees(self):
24        pass
25
26
27
```

Instruction 1

```
1 def remove_employee(self, employee_id):
2     """
3     Remove an employee from the HRManagementSystem.
4     :param employee_id: The employee's id, int.
5     :return: If the employee is already in the HRManagementSystem,
6     returns True, otherwise, returns False.
7     >>> hrManagementSystem = HRManagementSystem()
8     >>> hrManagementSystem.employees = {1: {'name': 'John', 'position':
9         'Manager', 'department': 'Sales', 'salary': 100000}}
10    >>> hrManagementSystem.remove_employee(1)
11    True
12    >>> hrManagementSystem.remove_employee(2)
13    False
14    """
```

Instruction 2

```
1 def remove_employee(self, employee_id):
2     """
3     From the HRManagementSystem, delete an employee
4     by specifying their 'employee_id'. First, determine
5     if this 'employee_id' is listed in the dictionary
6     of employees of the HRManagementSystem. If located,
7     expunge the employee from this dictionary and issue a
8     'True' to signal a successful operation. If the
9     'employee_id' is missing, issue a 'False', signaling
10    the operation's failure.
11    :param employee_id: The employee's id, int.
12    :return: If the employee is already in the HRManagementSystem,
13    returns True, otherwise, returns False.
14
15    """
```

Instruction 3

```
1 def remove_employee(self, employee_id):
2     """
3     Use the 'employee_id' to exclude an employee from the HRManagementSystem.
4     Assess whether the specified 'employee_id' exists within the dictionary
5     of employees. If it does, remove the respective employee from the directory
6     and return 'True' to denote success. If not, return 'False' to denote the
7     unsuccessful attempt.
8     :param employee_id: The employee's id, int.
9     :return: If the employee is already in the HRManagementSystem, returns True,
10    otherwise, returns False.
11    """
```

38. In a few words, what is the task this function aims to implement? *

39. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ☐ Very Easy
- ☐ Easy
- ☐ Not Easy Nor Difficult
- ☐ Difficult
- ☐ Very Difficult

40. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- ☐ 1 min
- ☐ 5 min
- ☐ 10 min
- ☐ 20 min
- ☐ 40 min
- ☐ > 40 min

41. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 11:

Below are instructions for implementing the function

Instruction 1

```
1 def get_odd_collatz(n):
2     """
3     Given a positive integer n, return a sorted list that has the odd numbers
4     in collatz sequence.
5
6     The Collatz conjecture is a conjecture in mathematics that concerns a
7     sequence defined as follows: start with any positive integer n. Then
8     each term is obtained from the previous term as follows: if the previous
9     term is even, the next term is one half of the previous term. If the
10    previous term is odd, the next term is 3 times the previous term plus 1.
11    The conjecture is that no matter what value of n, the sequence will always
12    reach 1.
13
14    Note:
15        1. Collatz(1) is [1].
16        2. returned list sorted in increasing order.
17
18    For example:
19    get_odd_collatz(5) returns [1, 5]
20    # The collatz sequence for 5 is [5, 16, 8, 4, 2, 1],
21    so the odd numbers are only 1, and 5.
22    """
```

Instruction 2

```
1 def get_odd_collatz(n):
2     """
3     Write a function named 'get_odd_collatz' that takes a positive integer n
4     and returns a sorted list of odd numbers from the Collatz sequence starting
5     at n. The Collatz sequence is defined such that each term is obtained from
6     the previous term as follows: if the term is even, the next term is half of
7     the current term; if the term is odd, the next term is three times the current
8     term plus one. The sequence is believed to always reach 1 regardless of the
9     starting number.
10    """
```

Instruction 3

```
1 def get_odd_collatz(n):  
2     """  
3     Program a function 'get_odd_collatz' which accepts a single positive  
4     integer 'n' and outputs a sorted array of the odd numbers found in the  
5     Collatz sequence starting from 'n'. This sequence continues with 'n'  
6     and iteratively conducts the rule: 'x' becomes 'x // 2' if even,  
7     else 'x * 3 + 1' if odd. The function employs an empty list 'ans' to  
8     gather odd numbers and 'x' is set as 'n'. It loops until 'x' matches 1,  
9     appending to 'ans' when 'x' is odd through the check 'x % 2 == 1',  
10    subsequently adjusting 'x' on its variation to even or odd. When the  
11    series ends with the addition of 1 to 'ans', the function finishes by r  
12    eturning this list in an ascending sort.  
13    """
```

42. In a few words, what is the task this function aims to implement? *

43. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ☐ Very Easy
- ☐ Easy
- ☐ Not Easy Nor Difficult
- ☐ Difficult
- ☐ Very Difficult

44. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- ☐ 1 min
- ☐ 5 min
- ☐ 10 min
- ☐ 20 min
- ☐ 40 min
- ☐ > 40 min

45. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 12:

Below are instructions for implementing the function:

Context

```
1 class Hotel:
2     """
3     This is a class as hotel management system, managing the booking,
4     check-in, check-out, and availability of rooms in a hotel with
5     different room types.
6     """
7
8     def __init__(self, name, rooms):
9         """
10        Initialize the three fields in Hotel System.
11        name is the hotel name.
12        available_rooms stores the remaining rooms in the hotel
13        booked_rooms stores the rooms that have been booked and
14        the person's name who booked rooms.
15        >>> hotel.name
16        'peace hotel'
17        >>> hotel.available_rooms
18        available_rooms = {'single': 5, 'double': 3}
19        >>> hotel.booked_rooms
20        {'single': {'guest 1': 2, 'guest 2': 1}, 'double': {'guest1': 1}}
21        """
22        self.name = name
23        self.available_rooms = rooms
24        self.booked_rooms = {}
25
26    def book_room(self, room_type, room_number, name):
27        pass
28
29    def check_out(self, room_type, room_number):
30        pass
31
32    def get_available_rooms(self, room_type):
33        pass
```

Instruction 1

```
1 def check_in(self, room_type, room_number, name):
2     """
3     Check if the room of the specified type and number is booked by
4     the person named name.
5     Remove this name when check in successfully(room_number is equal
6     to specific person's booked_rooms. When the actual check in quantity
7     (room_number) is less than the booked quantity, number in booked_rooms
8     will be booked quantity minus actual quantity
9     :param room_type: str, check in room type
10    :param room_number: int, check in room number
11    :param name: str, person name
12    :return False: only if the room_type is not in the booked_rooms or
13    room_number is higher than quantity in booked rooms.
14    >>> hotel = Hotel('peace hotel', {'single': 5, 'double': 3})
15    >>> hotel.book_room('single', 1, 'guest 1')
16    'Success!'
17    >>> hotel.check_in('single', 2, 'guest 1')
18    False
19    >>> hotel.check_in('single', 1, 'guest 1')
20    >>> hotel.booked_rooms
21    {'single': {}}
22    """
```

Instruction 2

```
1 def check_in(self, room_type, room_number, name):
2     """
3     Verify whether the room specified by 'room_type' and 'room_number'
4     is reserved under the guest 'name'. Start by checking if 'room_type'
5     is present in the 'self.booked_rooms' dictionary. If it is missing,
6     return 'False' right away. If it is found, see if 'name' is linked
7     to any reservations within that category. Should the 'room_number'
8     match the exact figure booked, delete that entry for 'name' from
9     'self.booked_rooms'. If the figure attempting to be checked in is
10    smaller, deduct that count from the total reserved for 'name',
11    adjusting the booked records in 'self.booked_rooms' accordingly.
12    If 'room_number' surpasses the amount booked under 'name', return
13    'False', signifying an overcheck attempt.
14    :param room_type: str, check in room type
15    :param room_number: int, check in room number
16    :param name: str, person name
17    :return False: only if the room_type is not in the booked_rooms or
18    room_number is higher than quantity in booked rooms.
19    """
```

Instruction 3

```
1 def check_in(self, room_type, room_number, name):
2     """
3     In the 'Hotel' class, verify if a room specified by 'room_type' and
4     'room_number' is reserved under the 'self.booked_rooms' dictionary
5     by a customer called 'name'. Start by ascertaining if 'room_type'
6     is a key within 'self.booked_rooms'. If absent, respond with 'False'.
7     If present, proceed to check if 'name' has reservations for this type
8     of room. If records for 'name' exist, and the 'room_number' aligns with
9     the number reserved under 'name', execute the 'pop' method to delete this
10    record from 'self.booked_rooms[room_type]'. If 'room_number' is smaller
11    than the reserved amount, reduce the booked quantity for 'name' by
12    'room_number' using subtraction assignment. Issue 'False' if 'room_number'
13    exceeds what 'name' had booked, suggesting overbooking.
14    :param room_type: str, check in room type
15    :param room_number: int, check in room number
16    :param name: str, person name
17    :return False: only if the room_type is not in the booked_rooms or room_number
18    is higher than quantity in booked rooms.
19    """
```

46. In a few words, what is the task this function aims to implement? *

47. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ☐ Very Easy
- ☐ Easy
- ☐ Not Easy Nor Difficult
- ☐ Difficult
- ☐ Very Difficult

48. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- ☐ 1 min
- ☐ 5 min
- ☐ 10 min
- ☐ 20 min
- ☐ 40 min
- ☐ > 40 min

49. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Demographics Information

50. What is your highest education qualification? *

Une seule réponse possible.

- ☐ Ph.D.
- ☐ Master's Degree
- ☐ Bachelor Degree
- ☐ Autre :

51. What is your field of work? (Engineer, Researcher, Developer, etc.) *

52. How many years of development experience do you have? *

Une seule réponse possible.

- ☐ Less than 5
- ☐ Between 5 and 10
- ☐ More than 10

53. Thank you for completing the survey! Any feedback you would like to provide?

Ce contenu n'est ni rédigé, ni cautionné par Google.

Google Forms

